

Chataalk

실시간 채팅 서비스 프로젝트

WebSocket/STOMP 기반 실시간 메시징부터
읽음 정합성, 재연결 복구, Redis Write-Back,
파일 처리, 메시지 검색 UX까지 직접 설계하고 구현한
개인 프로젝트입니다.

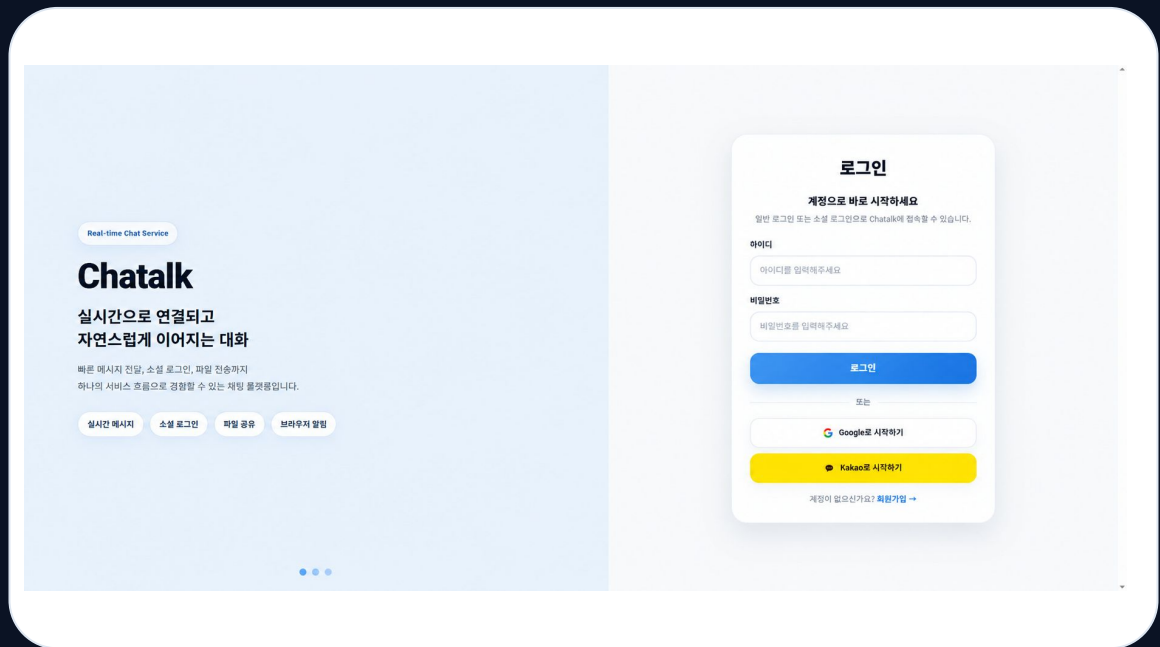
Java 21

Spring Boot

Vue

Redis

Docker



박민수 | Java/Spring Backend
배포 URL: <https://chataalk.store>
Repository: github.com/minsu11/live_chat

실제 배포 서비스 화면 기반

01. 프로젝트 한눈에 보기

단순 채팅 UI가 아니라, 실시간 서비스에서 실제로 문제가 되는 메시지 유실, 읽음 정합성, DB update 경합, Redis 장애, 파일 orphan, 검색 UX를 중심으로 설계한 프로젝트입니다.

46/60 → 60/60

동일 채팅방 DB 저장

300명

high 시나리오 연결

3개 저장소

API / Auth / Front

진행 중

2025.12 ~ 진행 중

핵심 구현 범위

- WebSocket/STOMP 기반 실시간 메시지 송수신
- 사용자별 lastReadMessageId 기반 읽음/안읽음 정합성
- afterMessageId 기반 재연결 후 누락 메시지 복구
- Redis Pub/Sub + Write-Back + Dirty Set + Bulk Update
- 파일 선업로드와 orphan attachment cleanup

담당 역할

- API 서버: 채팅방, 메시지, 읽음, 파일, 검색 도메인 설계/구현
- Auth 서버: 일반 로그인, OAuth2, JWT/쿠키 인증 처리
- Frontend: Vue 기반 채팅 UI, 검색 하이라이트, 스크롤 보정
- Infra: Docker 배포, Nginx Reverse Proxy, Cloudflare 연결
- 검증: clientMessageId 기반 publish/DB 저장/WebSocket 수신 비교

02. 프로젝트가 해결하려 한 문제

1

메시지 유실

WebSocket 연결이 끊긴 동안 발생한 메시지를 어떻게 복구할 것인가?

afterMessageId 기반 catch-up API

2

읽음 정합성

채팅방 unread와 메시지별 unread는 기준이 다른데 어떻게 분리할 것인가?

lastReadMessageId / unreadCount 분리

3

DB update 경합

메시지 발송마다 lastMessage, unread를 DB에 update하면 row 경합이 커진다.

Redis Write-Back + Bulk Update

4

Redis 장애

Redis 쓰기 실패 시 DB fallback, AOF 기반 복구 전략 보완

Circuit Breaker fallback + AOF 설정

5

파일 orphan

파일 업로드 후 메시지를 보내지 않으면 미연결 첨부 데이터가 남는다.

orphan attachment cleanup scheduler

6

검색 UX

검색 결과 목록만으로는 대화 흐름과 위치 이동이 불편하다.

context fetch + highlight + scrollTop 보정

03. 기술 스택과 저장소 구성

기술 스택

Backend

Java 21, Spring Boot, Spring Security, JPA, Querydsl

Realtime

WebSocket, STOMP, SockJS

Data

MySQL, Redis

Frontend

Vue.js

Infra

Docker, Nginx, Cloudflare, GitHub Actions

Resilience

Resilience4j Circuit Breaker, Redis AOF 설정

Repository 역할 분리

API Server

채팅 도메인, WebSocket 처리, Redis/DB 동기화, 파일 처리, Redis Pub/Sub + Write-Back

github.com/minus11/live_chat

Front Repository

Vue 기반 채팅 UI, WebSocket 연결, 메시지 렌더링, 검색 UI

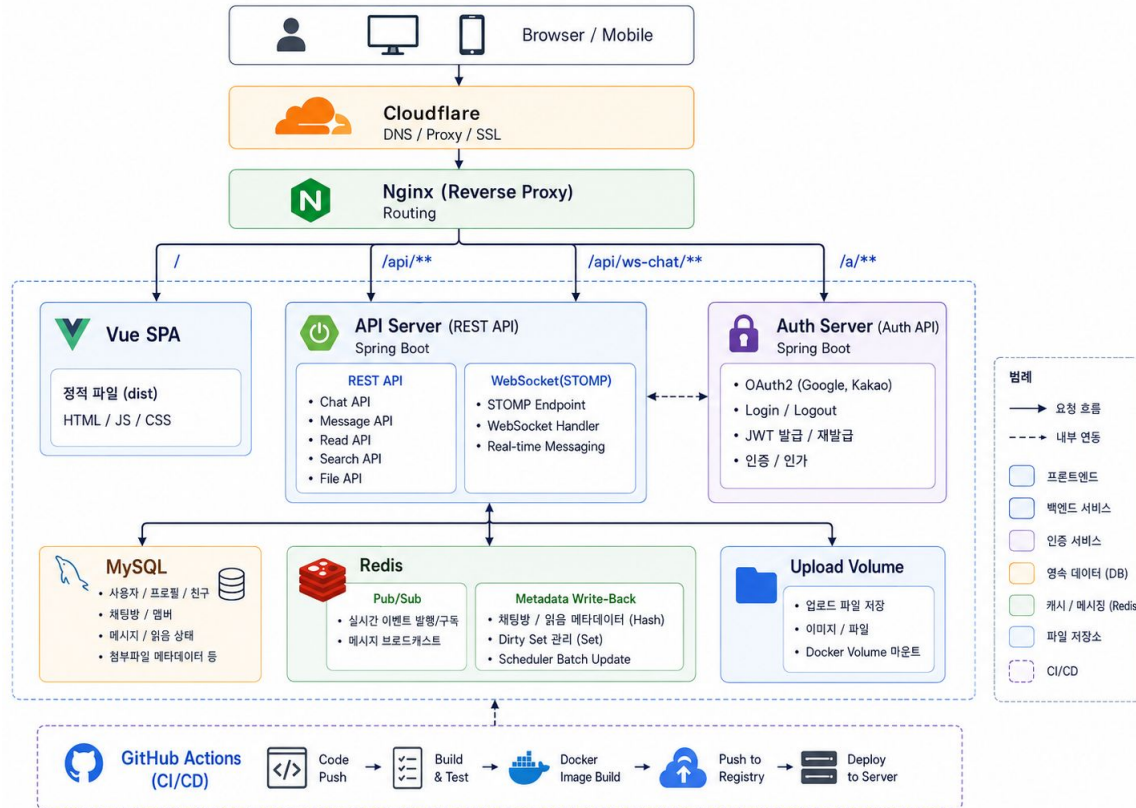
github.com/minus11/live_chat_front

Auth Server

로그인, OAuth2, 토큰 발급/재발급, 인증 처리

github.com/minus11/live_chat_auth

04. 전체 배포 아키텍처



실제 배포 기준 전체 아키텍처

요청 라우팅 기준

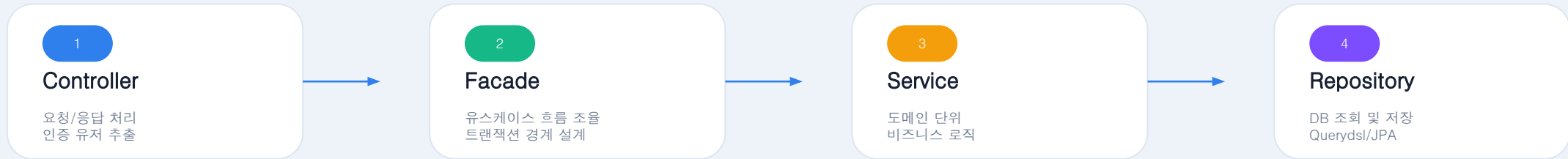
- **/** : Vue SPA dist 정적 파일 서빙
- **/api/**, /api/ws-chat/**** : API 서버로 프록시
- **/a/**** : Auth 서버
- JWT/HttpOnly Cookie 기반 인증 유지

User → Cloudflare → Nginx
 /api → API Server
 /a → Auth Server
 /api/ws-chat → API Server(WebSocket/STOMP)

API Server → MySQL / Redis / Upload Volume

05. API 서버 설계 구조

Controller가 모든 흐름을 직접 처리하지 않고, Facade가 하나의 유스케이스를 조율하도록 계층을 분리했습니다.



메시지 전송 Facade에서 조율한 흐름

- 채팅방 멤버 검증 및 메시지 타입 검증
- 메시지 저장 후 첨부파일 연결
- 읽음/안읽음 메타데이터 갱신
- 수신자별 응답 DTO 생성 후 WebSocket 브로드캐스트

설계 의도

메시지 저장, 첨부파일 연결, 읽음 처리, 브로드캐스트가 하나의 요청 흐름 안에서 이어지기 때문에 Controller에 모두 넣으면 코드가 비대해집니다. Facade 계층을 두어 흐름을 명확히 하고, Service는 도메인 로직에 집중하도록 분리했습니다.

Core Features

실시간 메시징, 인증, 파일, 검색 기능을 실제 서비스 흐름 기준으로 구성했습니다.

WebSocket

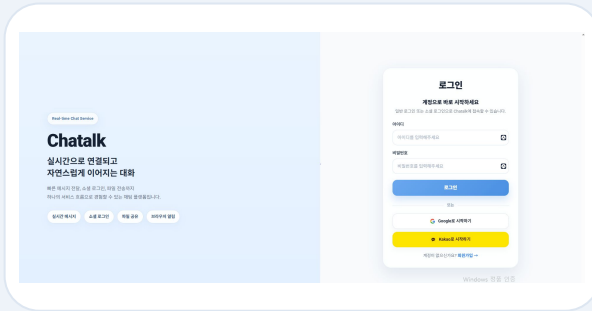
OAuth2

File

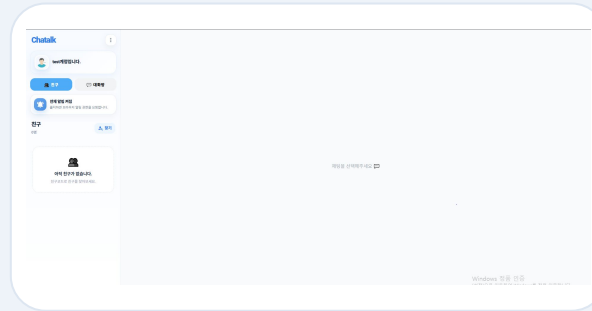
Search

06. 실제 서비스 시연 흐름

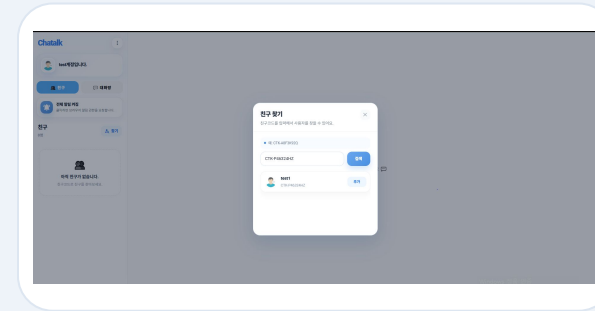
로그인부터 친구 검색, 1:1 채팅, 파일 전송, 메시지 검색까지 실제 배포 서비스 화면을 기준으로 정리했습니다.



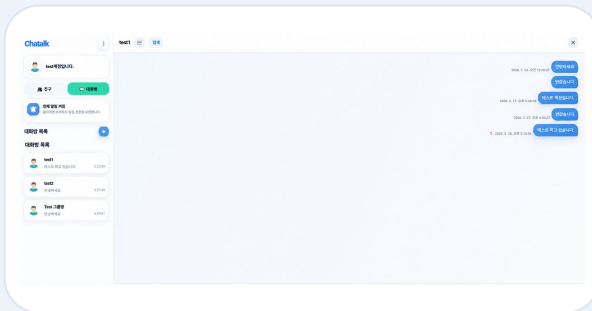
로그인 / OAuth2



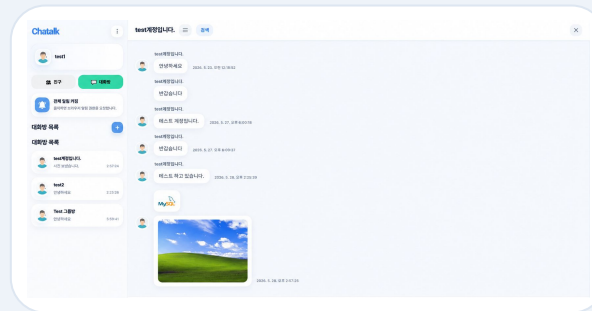
내 프로필 / friendCode



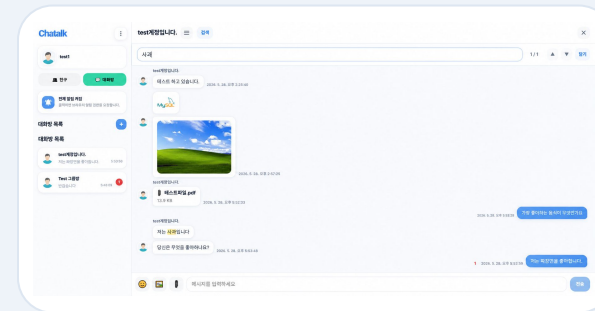
friendCode 친구 검색



1:1 실시간 채팅

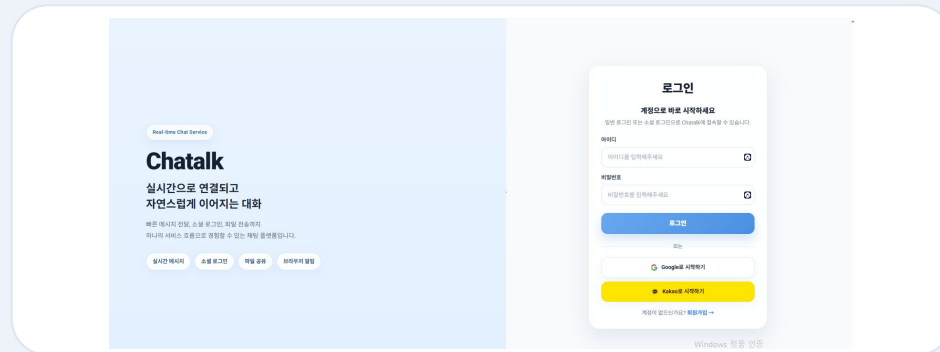


이미지 메시지

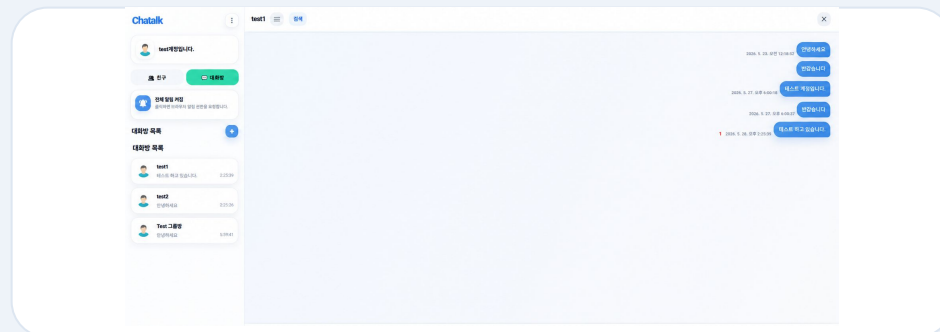


검색 하이라이트

07. Auth 서버 분리와 OAuth2 로그인



일반 로그인 + Google/Kakao OAuth2 진입



OAuth2 인증 성공 후 내부 사용자 매핑 흐름

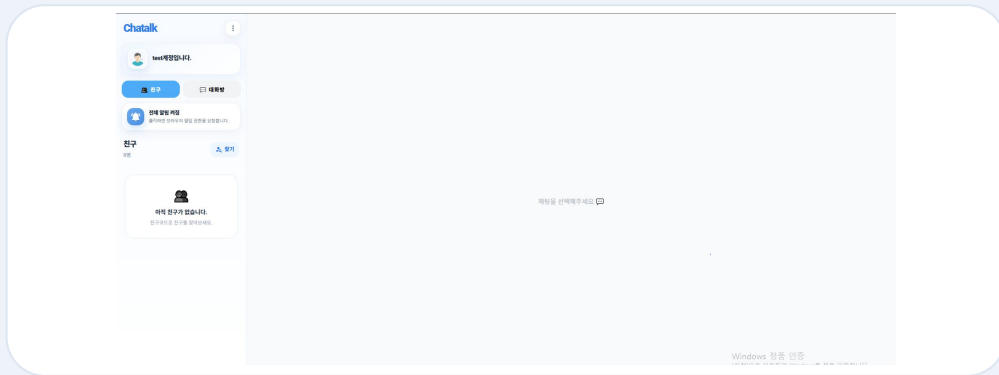
구현 핵심

- Auth 서버를 분리하여 로그인, OAuth2, 토큰 발급/재발급 책임을 분리
- OAuth2 성공 시 provider + providerUserId 기준으로 oauth_account 조회
- 기존 계정이 없으면 user, user_profile, oauth_account 자동 생성
- 내부 uuid 기준으로 JWT 발급 후 HttpOnly Cookie 전달
- 이메일이 없는 Kakao 계정도 안정적으로 재로그인 가능하도록 설계

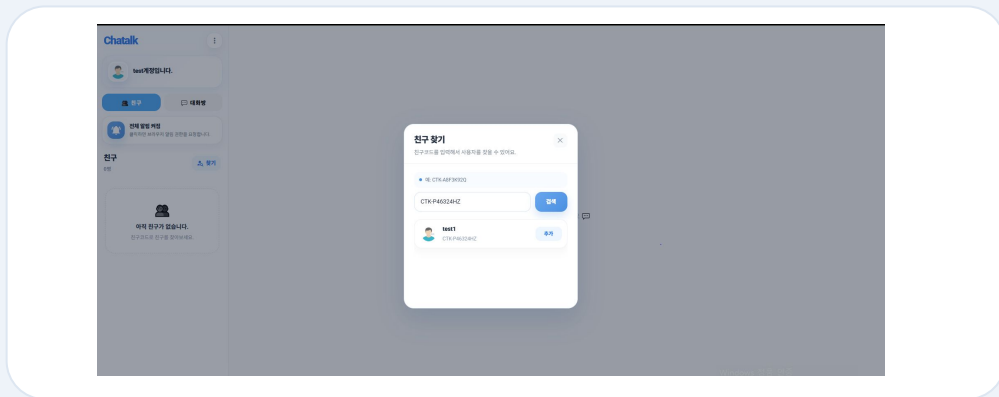
OAuth2 인증 성공

-> provider/providerUserId 추출
 -> oauth_account 조회
 -> user/user_profile 자동 생성
 -> uuid 기준 JWT 발급
 -> HttpOnly Cookie 전달

08. OAuth 사용자를 고려한 friendCode 기반 친구 검색



내 프로필에서 friendCode 확인



friendCode로 사용자 검색 및 친구 추가

문제

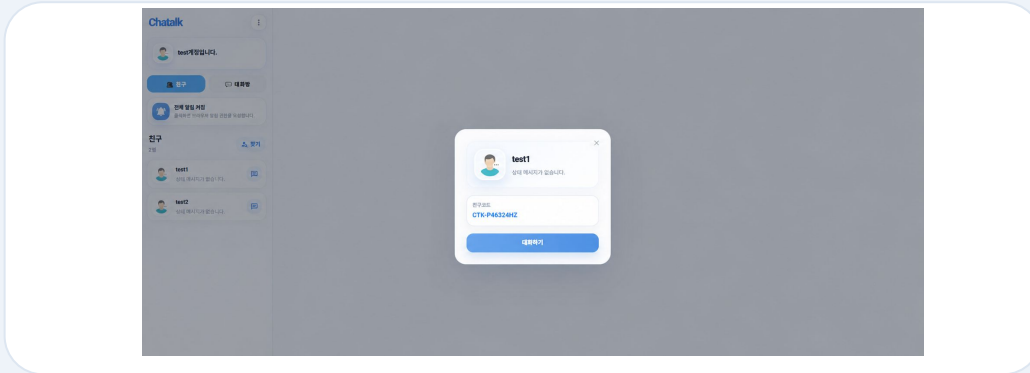
일반 로그인 사용자의 inputId는 공유 가능하지만, OAuth 사용자의 inputId는 시스템 내부에서 생성되기 때문에 친구 검색용 식별자로 적합하지 않았습니다.

해결

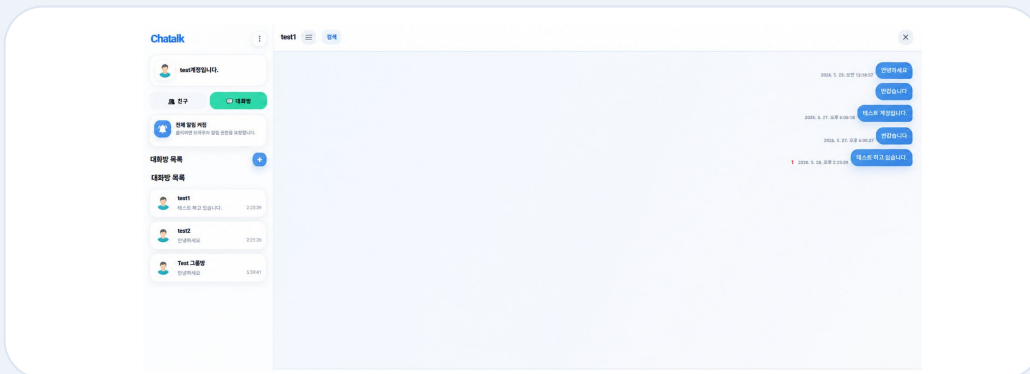
- 모든 사용자에게 공개 검색용 friendCode 부여
- 친구 검색은 friendCode 기준으로 수행
- 친구 등록, 1:1 채팅방 생성, 프로필 조회는 uuid 기준 유지

userId : 서버 내부 PK
uuid : API 대상 식별자
inputId : LOCAL 로그인 ID
friendCode : 공개 검색용 코드

09. 실시간 채팅방과 메시지 송수신



채팅방 목록



발신자 기준 채팅방

WebSocket/STOMP 메시지 처리 흐름

STOMP publish

멤버 검증

메시지 DB 저장

Redis 메타데이터 갱신

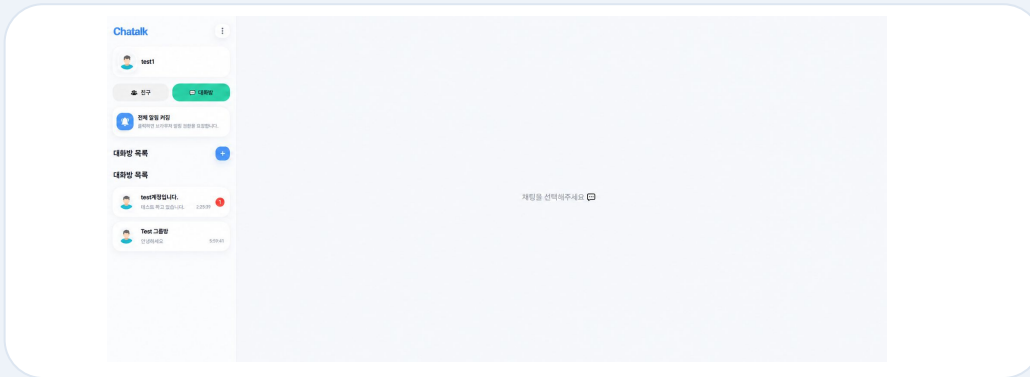
Redis Publish

Redis Subscriber

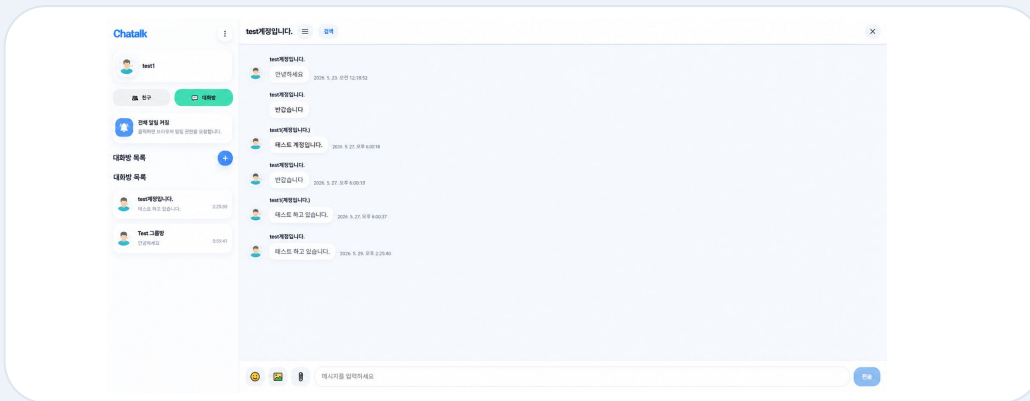
STOMP 브로드캐스트

서버는 메시지를 DB에 저장한 뒤 채팅방 참여자에게 사용자별 구독 경로로 메시지를 전송합니다. 메시지 타입을 TEXT, EMOJI, IMAGE, FILE, SYSTEM으로 분리하여 저장 방식, 미리보기, 렌더링, 검색 대상 여부를 명확히 구분했습니다.

10. 읽음/안읽음 정합성 설계



수신자 채팅방 목록 unread count



수신자 기준 채팅방

unread는 두 가지 의미를 가짐

채팅방 목록 unread

내가 아직 읽지 않은 메시지 수

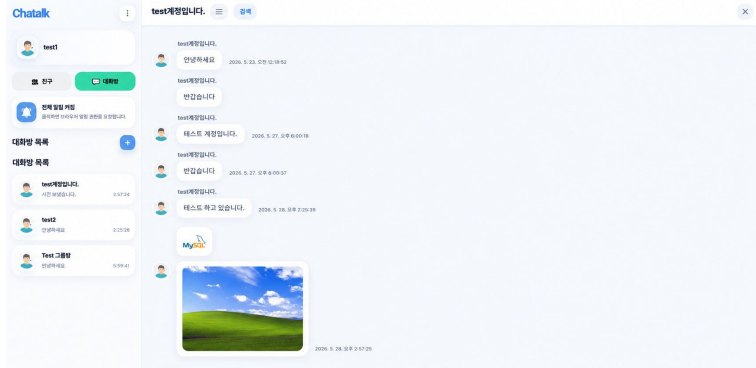
메시지별 unread

해당 메시지를 아직 읽지 않은 멤버 수

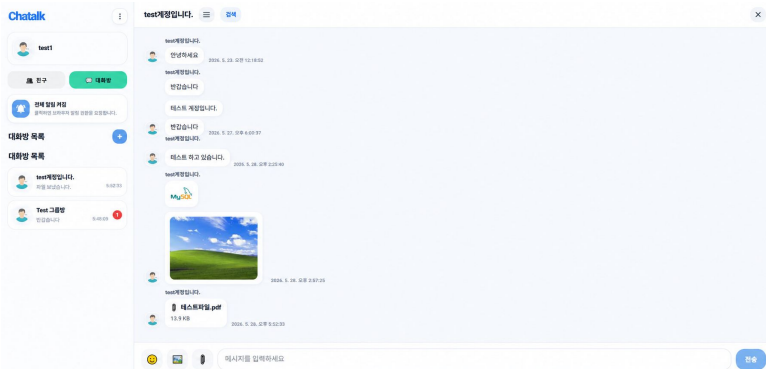
해결 방식

- 사용자별 lastReadMessageId로 마지막 읽은 위치 관리
- 채팅방 목록 unreadCount는 현재 사용자 기준으로 분리
- READ_UPDATED 이벤트로 메시지별 unread count 부분 갱신
- 메시지마다 읽음 row를 저장하지 않아 데이터 증가를 줄임

11. 이미지/파일 메시지와 orphan cleanup



이미지 메시지 전송 화면



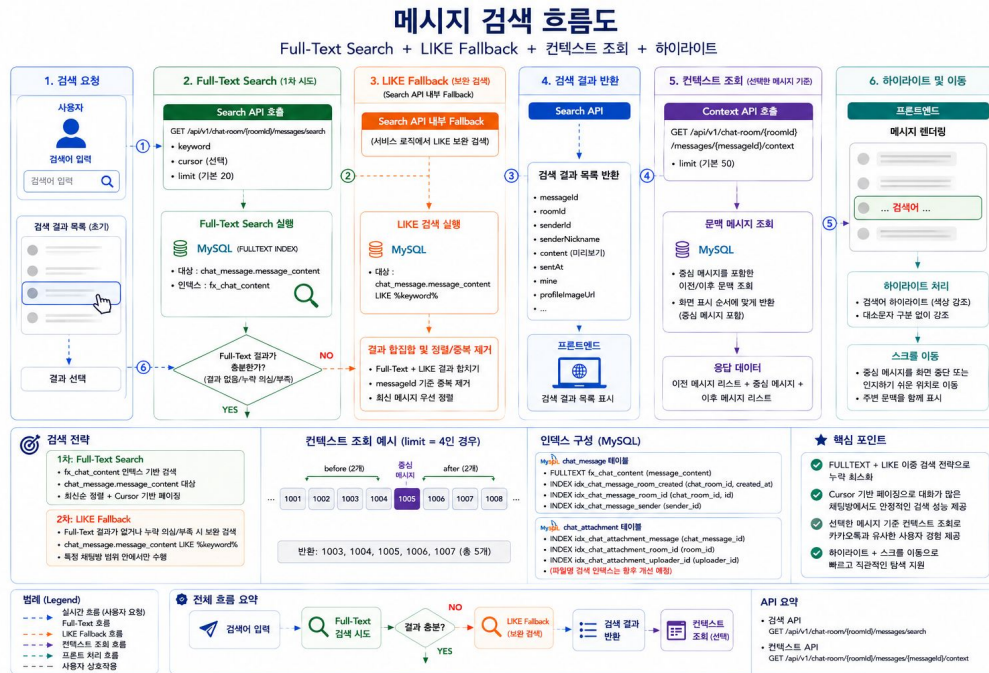
파일 메시지 전송 화면

파일 처리 구조

- 파일을 메시지와 동시에 저장하지 않고 먼저 첨부파일로 선업로드
- 전송 확정 시 FILE/IMAGE 메시지를 저장하고 attachment와 message 연결
- 업로드 후 전송 취소 시 message와 연결되지 않은 orphan attachment 발생 가능
- 스케줄러가 일정 시간이 지난 미연결 첨부 데이터와 실제 파일을 정리

1. 파일 선택
2. 첨부파일 선업로드
3. attachment metadata 저장
4. 메시지 전송 확정
5. FILE/IMAGE 메시지 저장
6. attachment-message 연결
7. orphan cleanup scheduler

12. 채팅 메시지 검색 UX

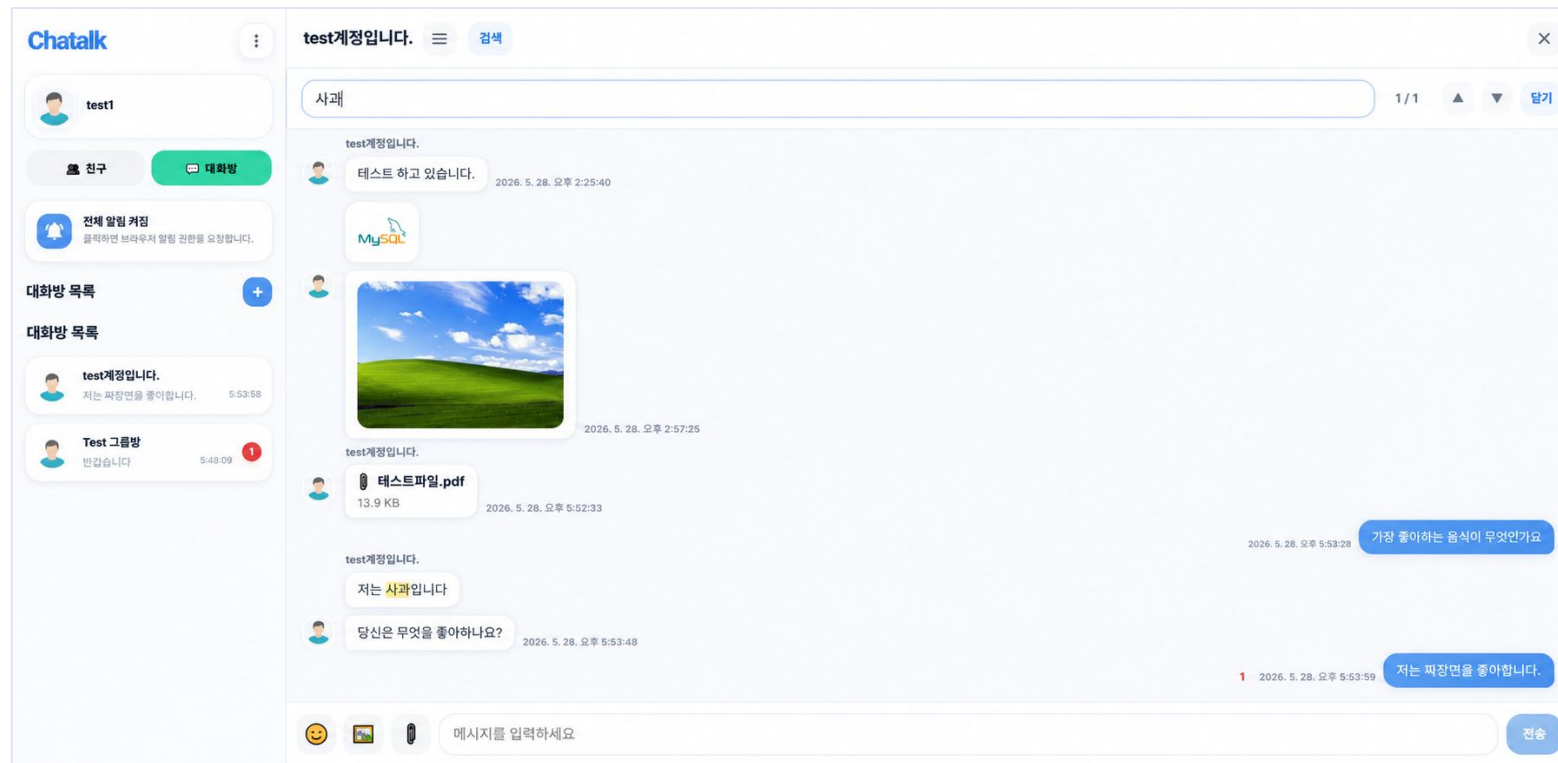


검색 API와 context API 분리

- 채팅방 단위 키워드 검색 결과를 최신순 cursor 기반으로 반환
- 검색 결과 선택 시 targetMessageId 기준 앞뒤 대화 context 조회
- Full-Text Search를 기본으로 사용하되 LIKE fallback으로 누락 보완
- 프론트에서 검색어 하이라이트와 messages 컨테이너 기준 scrollTop 보정 적용

검색어 입력
→ keyword search API
→ 검색 결과 cursor 반환
→ 선택한 messageId 기준 context API
→ 하이라이트
→ scrollTop 직접 보정

12-2. 검색 결과 하이라이트 시연



검색어 하이라이트와 검색 결과 이동

Reliability & Data Consistency

메시지 유실, DB 경합, Redis 장애 상황을 고려해 안정성을 보강했습니다.

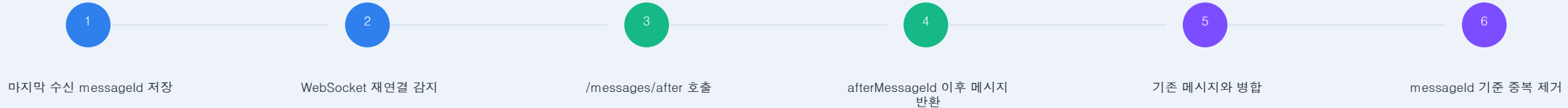
Catch-up

Write-Back

Circuit Breaker

13. WebSocket 재연결 후 누락 메시지 복구

WebSocket은 연결이 끊긴 동안 발생한 이벤트를 자동 보장하지 않기 때문에, 실시간 전송과 유실 복구의 책임을 분리했습니다.



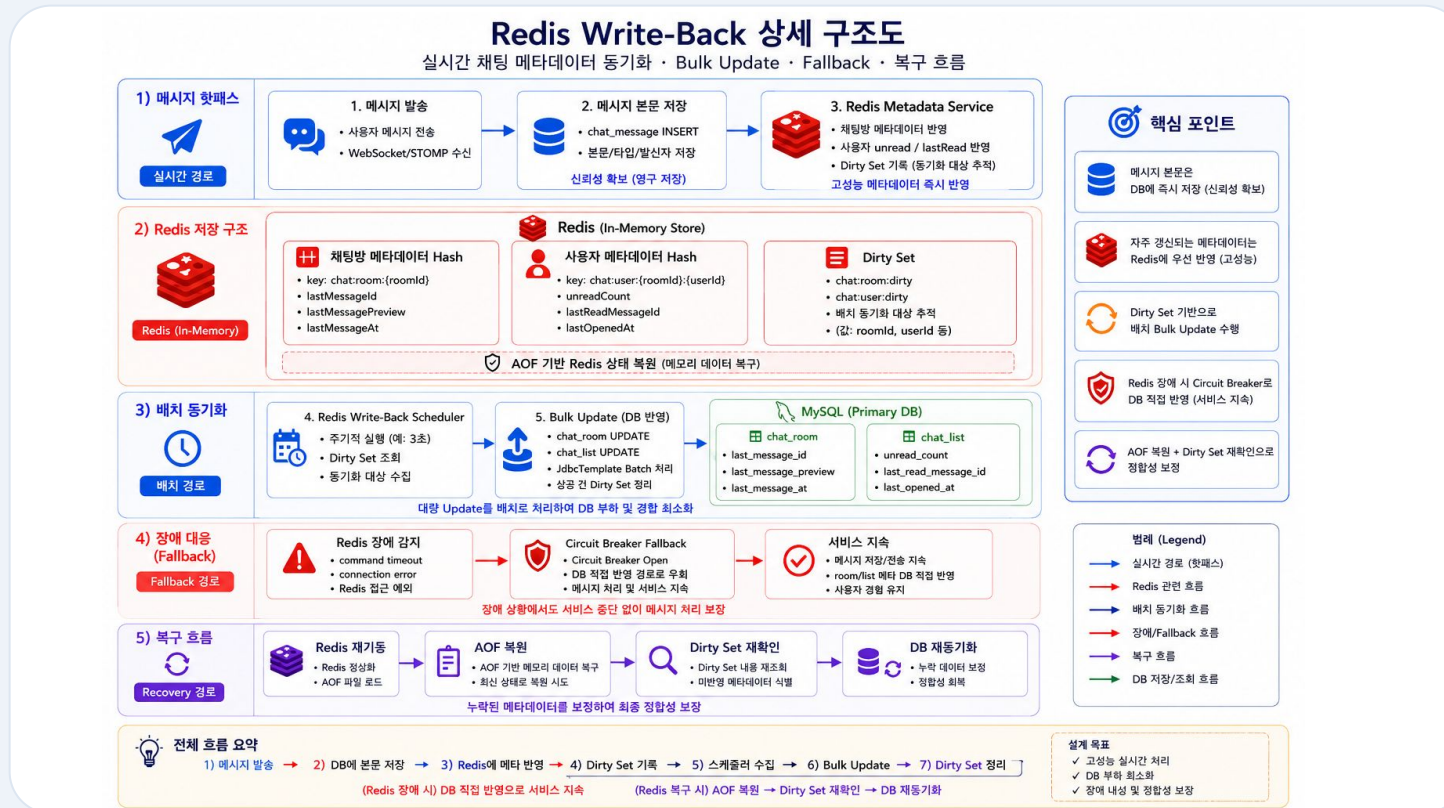
설계 포인트

- 실시간 전송은 WebSocket이 담당
- 연결 끊김 중 누락 구간은 HTTP API가 담당
- 클라이언트는 messageId 기준으로 중복 제거

효과

WebSocket 이벤트가 유실되더라도 마지막 수신 메시지 이후의 데이터를 HTTP로 재조회하여 메시지를 복구할 수 있습니다. 실시간성과 복구 가능성을 분리해 안정성을 높였습니다.

14. Redis Write-Back 기반 메타데이터 동기화



Redis Write-Back 상세 구조도

왜 Redis Write-Back을 적용했나

- 메시지 본문은 INSERT 중심이지만, 채팅방 목록 메타데이터는 UPDATE 중심
- 같은 채팅방에 메시지가 집중되면 lastMessage와 unread 업데이트 경합 가능
- 핫패스에서는 Redis Hash에 우선 반영하고 Dirty Set에 변경 대상 기록
- 스케줄러가 Dirty Set을 읽어 DB Bulk Update 수행

동일 조건 비교

sync-db: 60건 전송 → 56건 수신 / DB 46건 저장

Write-Back: 3회 반복 모두 60건 수신

DB·Redis 확인 실행: DB 60/60, Deadlock 증가량 0, Dirty Set 0
E2E p99: 7,945.15ms → 1,482.31ms

- sync-db는 문제 재현 실행 1회, Write-Back 지연시간은 3회 반복 결과의 중앙값 기준

Post-send tail: 6,459ms → 937ms

적용 효과: 메시지 요청 경로에서 메타데이터 DB UPDATE를 분리해 동일 row 경합을 완화하고, DB 반영 비용을 Batch Update로 분산했습니다.

15. Redis 메타데이터 장애 대응 및 복구 흐름

Redis 장애 Fallback 검증 로그

Redis 장애 감지 · Circuit Breaker 우회 · 복구 후 재동기화 확인

1 장애 감지 및 Fallback

```
02:23:36.405 [clientInboundChannel-19] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:23:36.407 [clientInboundChannel-19] DEBUG o.s.orm.jpa.JpaTransactionManager - Found thread-bound EntityManager [SessionImpl(1765696680<open>)] for JPA transaction
02:23:36.407 [clientInboundChannel-19] DEBUG o.s.orm.jpa.JpaTransactionManager - Participating in existing transaction
1 02:23:36.407 [clientInboundChannel-19] WARN c.c.r.s.ChatMetadataRedisService - [CircuitBreaker] Redis 장애! 유저(1, 방:2) 읽음 처리로 DB에 직접 반영합니다.
02:23:36.429 [lettuce-eventExecutorLoop-1-19] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:23:37.594 [lettuce-eventExecutorLoop-1-3] INFO t.i.core.protocol.ConnectionWatchdog - Reconnecting, last destination was localhost/<unresolved>:6379
02:23:37.954 [lettuce-eventExecutorLoop-1-2] INFO t.i.core.protocol.ConnectionWatchdog - Reconnecting, last destination was localhost/<unresolved>:6379
2 02:23:37.954 [lettuce-nioEventLoop-6-11] WARN t.i.core.protocol.ConnectionWatchdog - Cannot reconnect to [localhost/<unresolved>:6379]: Connection refused: getsockopt: localhost/127.0.0.1:6379
02:23:37.954 [lettuce-nioEventLoop-6-12] WARN t.i.core.protocol.ConnectionWatchdog - Cannot reconnect to [localhost/<unresolved>:6379]: Connection refused: getsockopt: localhost/127.0.0.1:6379
02:23:39.437 [clientInboundChannel-19] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:23:39.438 [clientInboundChannel-19] DEBUG o.s.orm.jpa.JpaTransactionManager - Found thread-bound EntityManager [SessionImpl(1765696680<open>)] for JPA transaction
02:23:39.438 [clientInboundChannel-19] DEBUG o.s.orm.jpa.JpaTransactionManager - Participating in existing transaction
3 02:23:39.438 [clientInboundChannel-19] WARN c.c.r.s.ChatMetadataRedisService - [CircuitBreaker] Redis 장애! 채팅방(2) 메타데이터를 DB에 직접 저장합니다. 사유: Redis command timed out
```

2 배치 스케줄러 예외 확인

```
1 02:23:33.683 [MessageBroker-13] ERROR o.s.s.TaskUtils$LoggingErrorHandler - Unexpected error occurred in scheduled task
org.springframework.dao.QueryTimeoutException: Create breakpoint: Redis command timed out
    at org.springframework.data.redis.connection.Lettuce.LettuceExceptionConverter.convert(LettuceExceptionConverter.java:68)
    at org.springframework.data.redis.connection.Lettuce.LettuceExceptionConverter.convert(LettuceExceptionConverter.java:41)
    at org.springframework.data.redis.PassThroughExceptionTranslationStrategy.translate(PassThroughExceptionTranslationStrategy.java:40)
    at org.springframework.data.redis.FallbackExceptionTranslationStrategy.translate(FallbackExceptionTranslationStrategy.java:38)
    at org.springframework.data.redis.connection.Lettuce.LettuceConnection.convertLettuceAccessException(LettuceConnection.java:310)
    ... (중략) ...
    at com.chat.server.redis.scheduler.ChatMetadataBatchScheduler.flushScheduledData(ChatMetadataBatchScheduler.java:48)
    at com.chat.server.redis.scheduler.ChatMetadataBatchScheduler.flushMetadataToDb(ChatMetadataBatchScheduler.java:35)
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
```

3 복구 후 Batch 동기화 재개

```
1 02:25:50.882 [MessageBroker-11] INFO c.c.r.s.ChatMetadataBatchScheduler - [✓] 유저 메타데이터 1건 Bulk Update 완료
02:25:50.882 [MessageBroker-11] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:25:50.882 [MessageBroker-11] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:25:50.882 [MessageBroker-11] DEBUG o.s.jdbc.core.JdbcTemplate - Executing SQL batch update [UPDATE chat_list SET unread_count = ?, last_read_message_id = ?, last_opened_at = ? WHERE chat_room_id = ? AND user_id = ?]
02:25:50.883 [MessageBroker-11] DEBUG o.s.jdbc.core.JdbcTemplate - Executing prepared SQL statement [UPDATE chat_list SET unread_count = ?, last_read_message_id = ?, last_opened_at = ? WHERE chat_room_id = ? AND user_id = ?]
02:25:50.883 [MessageBroker-11] DEBUG o.s.jdbc.support.JdbcUtils - JDBC driver supports batch updates
2 02:25:50.885 [MessageBroker-11] INFO c.c.r.s.ChatMetadataBatchScheduler - [✓] 채팅방 메타데이터 1건 Bulk Update 완료
02:25:50.885 [MessageBroker-11] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:25:50.885 [MessageBroker-11] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
```

✓ 검증 결과 1 Redis 장애 시 메시지 본문 저장 및 서비스 처리 지속 2 메타데이터는 Circuit Breaker Fallback으로 DB 직접 반영 3 장애 구간에서 Redis 접근 실패 확인 4 Redis 복구 후 Batch 동기화가 재개되어 정상성 회복

문제

Redis Write-Back 구조에서는 최신 메타데이터가 Redis에 먼저 반영됩니다.

이 상태에서 Redis 장애가 발생하면 DB 미반영 데이터의 유실 가능성을 고려해야 했습니다.

대응

- Redis 메타데이터 쓰기 실패
Circuit Breaker fallback → DB 직접 반영
- DB Batch Update 실패
Dirty Key 유지 → 다음 Scheduler에서 재시도
- Redis AOF
장애 전 write 복구를 위한 영속화 기반

남은 과제

AOF 복구 후 DB 반영 순서와 멍등성 검증은 후속 과제로 분리했습니다.

Troubleshooting

기능 구현보다 문제의 원인, 해결 방식, 검증 기준이 드러나도록 정리했습니다.

DB Lock

Search

OAuth

Scroll

16. 주요 트러블슈팅 요약

동일 채팅방 메타데이터 UPDATE 경합

메시지 발송마다 chat_room/chat_list 업데이트가 집중됨

Redis Write-Back + Dirty Set + Bulk Update

Redis 장애 시 미반영 데이터 유실 위험

DB 반영 전 Dirty 데이터가 Redis에만 존재

Redis 쓰기 실패 → DB fallback
DB batch 실패 → Dirty Key 유지 후 재시도

읽음/안읽음 의미 혼동

채팅방 unread와 메시지별 unread 기준이 다름

lastReadMessageId와 unreadCount 역할 분리

WebSocket 재연결 중 메시지 유실

끊긴 동안 이벤트를 수신하지 못함

afterMessageId 기반 catch-up API

Full-Text Search 검색 누락

parser/token/stopword 영향으로 일부 키워드 누락

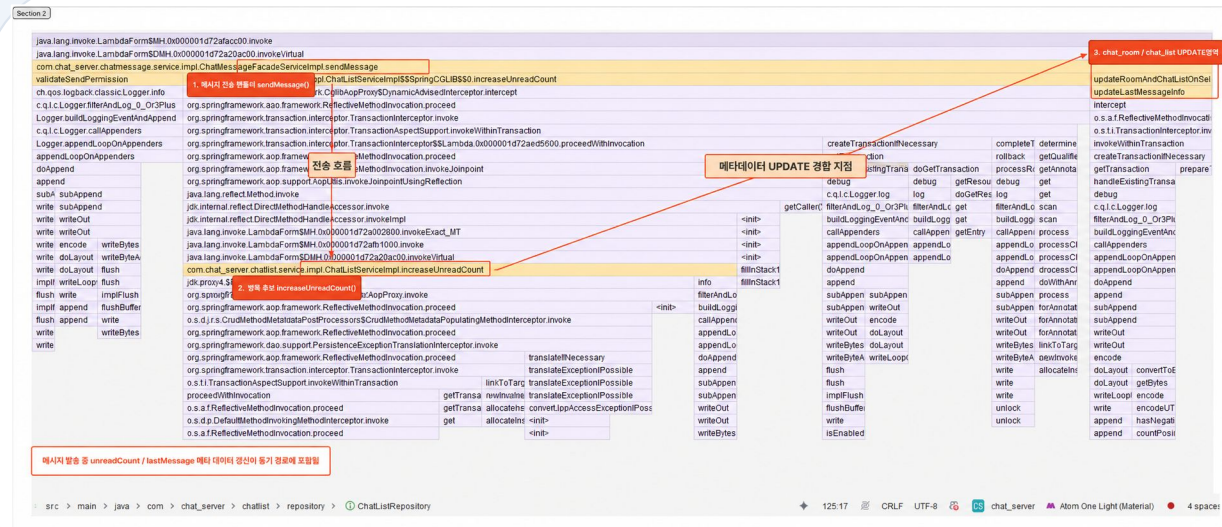
LIKE fallback 추가

검색 결과 스크롤 위치 불안정

scrollIntoView가 messages 컨테이너 구조와 맞지 않음

컨테이너 기준 scrollTop 직접 보정

17. DB UPDATE 경합을 Redis Write-Back으로 분리



문제 재현

- 3 VU × 20건 = 60건 동시 전송
- 수신 56/60 · DB 저장 46/60
- MySQL Deadlock · Rollback · HikariCP 포화

원인

- chat_room / chat_list 메타데이터 UPDATE가 요청 트랜잭션에 포함
- 동일 채팅방에 쓰기가 집중되며 Row Lock 경쟁과 잠금 순서 충돌 발생

해결

- Redis Hash + Dirty Set으로 요청 경로에서 DB UPDATE 분리
- Scheduler에서 JdbcTemplate Batch Update 수행

결과

- Write-Back 3회 모두 수신 60/60
- DB·Redis 검증 실행: DB 저장 60/60
- Deadlock 증가량 0 · Dirty Set 최종 0

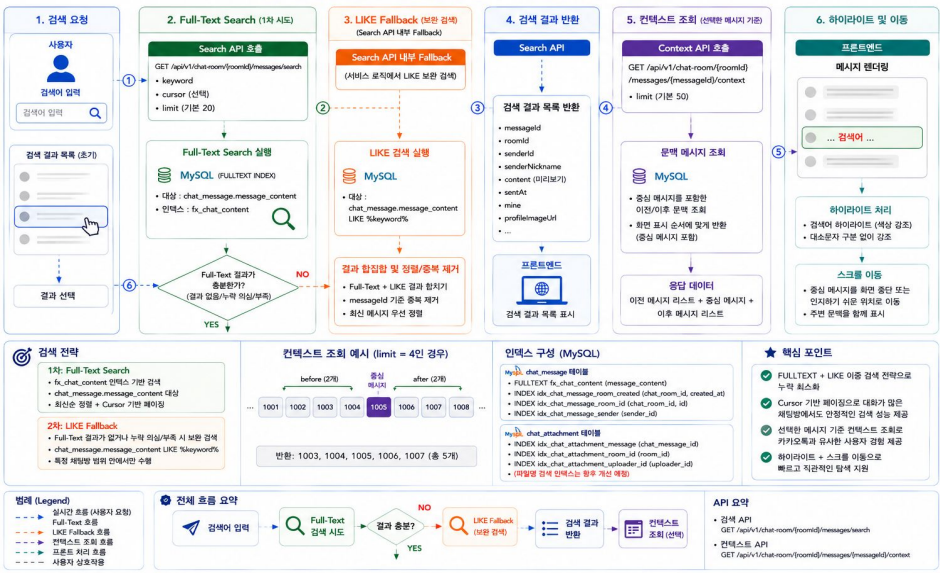
```
02:22:40.592 [MessageBroker-10] INFO c.o.r.s.ChatMetadataBatchScheduler - Redis -> DB 메타 데이터 동기화 시작
02:22:40.592 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:22:40.592 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:22:40.592 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:22:40.593 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:22:40.593 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:22:40.593 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:22:40.593 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Fetching Redis Connection from RedisConnectionFactory
02:22:40.593 [MessageBroker-10] DEBUG o.s.d.r.core.RedisConnectionUtils - Closing Redis Connection
02:22:40.594 [MessageBroker-10] DEBUG o.s.jdbc.core.JdbcTemplate - Executing SQL batch update [UPDATE chat_list SET unread_count = ?, last_read_message_id = ?, last_opened_at = ? WHERE chat_room_id = ? AND user_id = ?]
02:22:40.595 [MessageBroker-10] DEBUG o.s.jdbc.core.JdbcTemplate - Executing prepared SQL statement [UPDATE chat_list SET unread_count = ?, last_read_message_id = ?, last_opened_at = ? WHERE chat_room_id = ? AND user_id = ?]
02:22:40.596 [MessageBroker-10] DEBUG o.s.jdbc.support.JdbcUtils - JDBC driver supports batch updates
```

Profiler 및 batch log 기반 검증 자료

18. 메시지 검색 누락과 스크롤 위치 문제 해결

메시지 검색 흐름도

Full-Text Search + LIKE Fallback + 컨텍스트 조회 + 하이라이트



검색 누락 문제

MySQL Full-Text Search와 ngram parser를 적용했지만 일부 영문 키워드는 MATCH AGAINST에서 결과가 누락되었습니다. 채팅 검색에서는 랭킹보다 입력 문자열이 포함된 메시지를 빠짐없이 찾는 것이 중요하다고 판단했습니다.

해결

Full-Text Search를 유지하되 동일 API에서 LIKE fallback을 추가했습니다. 검색 범위는 채팅방 단위로 제한해 fallback 부담을 줄였습니다.

검색 API / context API 분리 흐름

스크롤 보정

채팅 화면은 전체 페이지가 아니라 messages 영역만 overflow-y: auto로 스크롤됩니다. 따라서 scrollToView() 대신 메시지 컨테이너와 대상 메시지의 위치를 직접 계산하여 scrollTop을 보정했습니다.

19. OAuth2 배포 문제와 운영 환경 정리

발생했던 문제

- Google 계정 선택 화면까지 이동하지 않고 로그인 화면으로 돌아옴
- client_id에 %22가 붙어 Google 401 invalid_client 발생
- callback redirect_uri가 localhost:9090으로 생성됨
- 운영 redirect host가 chataalk로 설정되어 .store가 누락됨

정리한 해결 기준

- Nginx /a/ proxy_pass는 auth-server 뒤에 /를 붙여 내부 경로를 보정
- client id/secret은 env에서 따옴표 없이 주입
- 운영 redirect-uri를
https://chataalk.store/a/login/oauth2/code/{registrationId} 형태로 명시
- SPRING_PROFILES_ACTIVE=prod와 OAuth env를 Docker run
--env-file로 주입

```
/a/oauth2/authorization/google
-> Google 계정 선택
-> /a/login/oauth2/code/google
-> Cookie 발급
-> https://chataalk.store 메인 진입
```


Verification & Load Test

clientMessageld 기준으로 publish, DB 저장, WebSocket 수신을 분리해 검증했습니다.

clientMessageld

DB 저장

수신 성공률

20. 부하 테스트 방식과 결과



21. 검증 자동화와 테스트 한계

초기 테스트의 문제

처음에는 클라이언트가 STOMP publish를 시도한 횟수만 측정해 서버가 실제 DB에 저장했는지, 구독자가 정상 수신했는지 구분하기 어려웠습니다.

테스트 코드 보강 결과

- JUnit 테스트 326개 / 실패 0개
- Line Coverage 64.36%
- Branch Coverage 65.23%
- JaCoCo Gate: Line $\geq 60\%$ / Branch $\geq 40\%$ → 통과

주요 검증

- Service / Facade / Controller
- JWT HTTP 인증 필터
- STOMP CONNECT 인증
- WebSocket Broadcaster
- Redis Scheduler / fallback / Dirty Key
- 파일 경계값 및 실패 처리

개선

- 모든 메시지에 clientMessageld 부여
- publish / DB 저장 / WebSocket 수신을 각각 비교
- 기대 수신 수 도달 또는 수신 증가 정지까지 drain 대기
- 중복과 누락을 메시지 단위로 검증

한계와 다음 개선

- 로컬 PC와 Node.js 부하 클라이언트 영향 가능
- 단일 서버, 단일 DB/Redis 기준 테스트
- 운영 환경의 네트워크, JVM, DB 커넥션 풀 튜닝 영향 미반영
- Micrometer / Prometheus / Grafana 기반 지표 수집 필요

22. 성과와 기술적 어필 포인트

1

실시간성

WebSocket/STOMP + Redis Pub/Sub 기반 실시간 이벤트
브로드캐스트 구현

2

정합성

읽음/안읽음 기준 분리, 재연결 후 누락 메시지 catch-up, 중복
제거

3

성능/경합 완화

Redis Write-Back 적용 후 동일 조건 DB 저장 46/60 → 60/60,
Deadlock 증가량 0 확인

4

장애 대응

Redis 쓰기 실패 DB fallback + batch 실패 Dirty Key 재시도

5

파일 처리

선업로드, 메시지 연결, orphan attachment cleanup 흐름 설계

6

검색 UX

Full-Text Search + LIKE fallback, context fetch, 하이라이트,
scrollTop 보정

23. 향후 개선 방향과 제출 링크

향후 개선 방향

- WebSocket 브로드캐스트 처리 비동기화와 fan-out 경로 최적화
- clientMessageId / idempotency key 기반 중복 전송 및 재시도 안정화
- OpenSearch/Elasticsearch 기반 검색 고도화
- Micrometer, Prometheus, Grafana 기반 관측성 강화
- 관리자 기능, 신고 처리, 운영 감사 로그 확장
- Redis Pub/Sub 다중 인스턴스 환경 검증

관련 링크

배포 URL	https://chataalk.store
API Server	https://github.com/minsu11/live_chat
Front Repository	https://github.com/minsu11/live_chat_front
Auth Server	https://github.com/minsu11/live_chat_auth

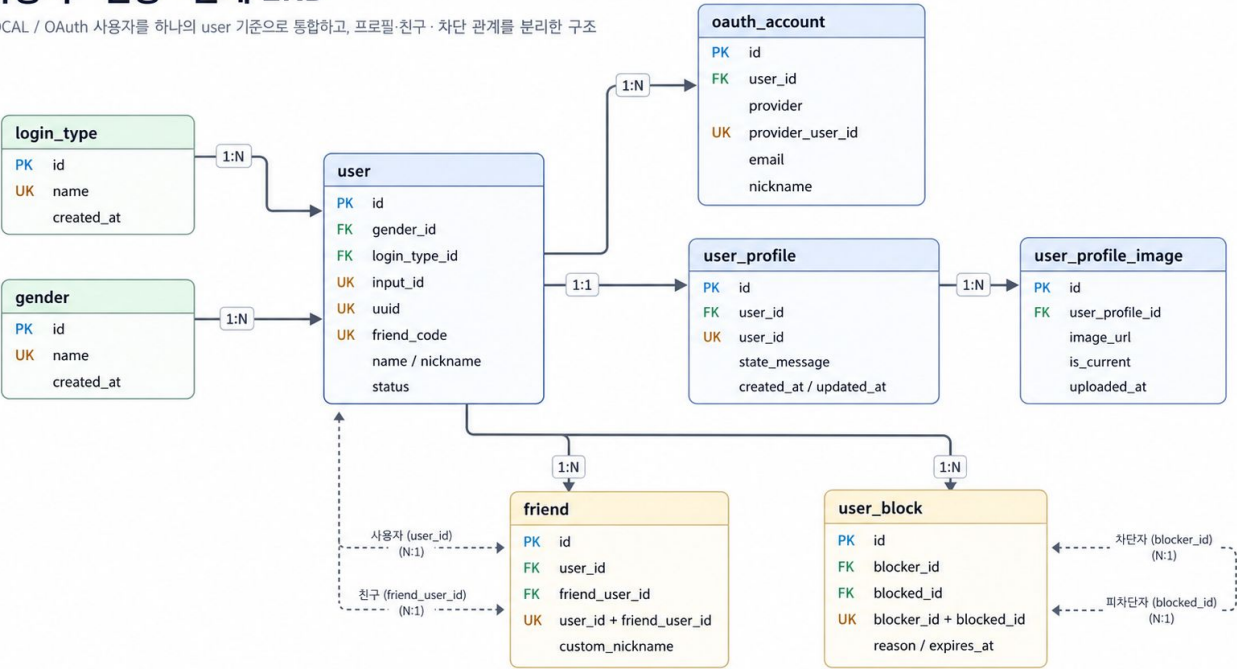
실제 서비스 화면, 아키텍처, 구현 의도, 트러블슈팅, 검증 결과가 한
흐름으로 보이도록 구성했습니다.

24. 사용자 · 인증 · 관계 ERD

사용자 식별자, OAuth 계정 매핑, 프로필, 친구/차단 관계를 분리한 구조입니다.

사용자 · 인증 · 관계 ERD

LOCAL / OAuth 사용자를 하나의 user 기준으로 통합하고, 프로필·친구·차단 관계를 분리한 구조



설계 포인트

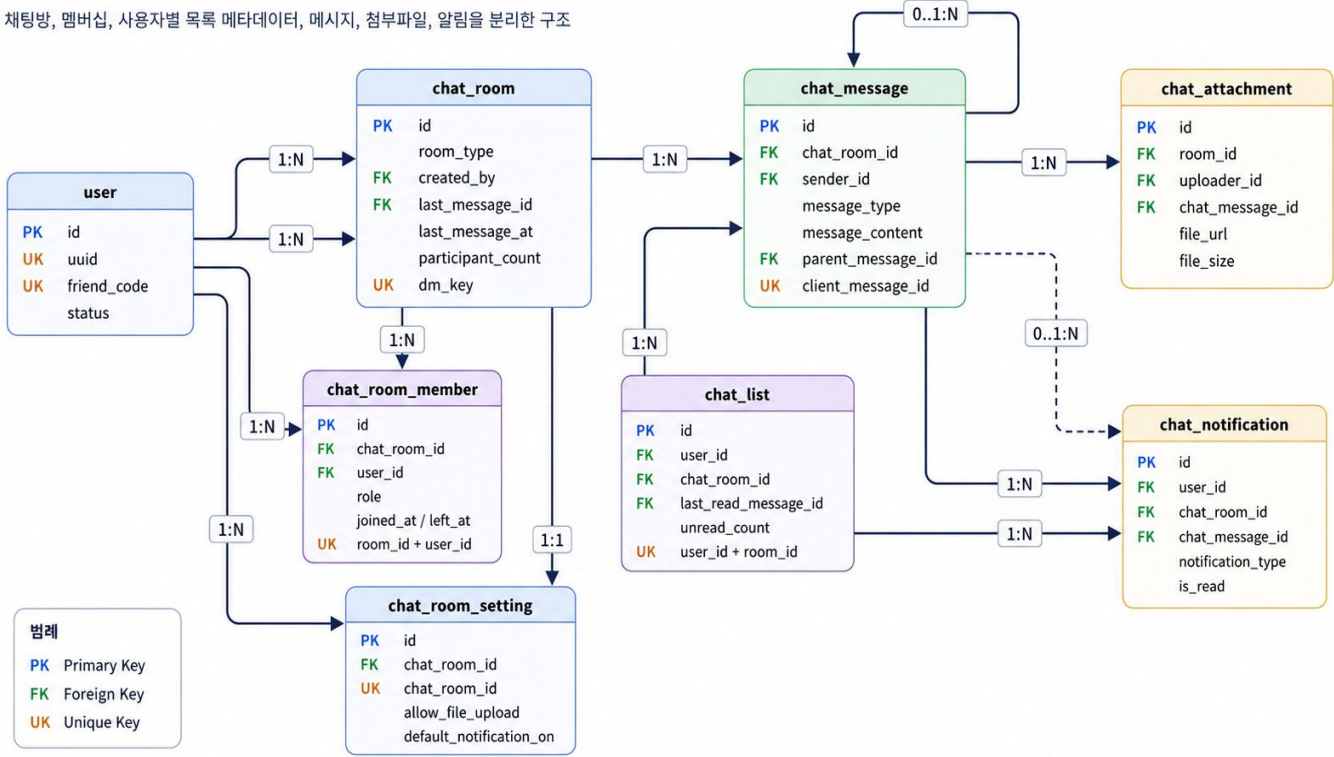
- 공개 식별자 분리: friendCode는 검색/공유용, uuid는 API 대상 식별자
- OAuth 안정성: email 대신 provider + providerUserId 조합으로 식별
- 프로필 확장: user_profile과 이미지 이력을 분리
- 관계 모델: friend / user_block은 방향성을 가진 사용자 관계

25. 채팅 · 메시지 · 파일 ERD

채팅방, 멤버십, 채팅방 목록 메타데이터, 메시지, 첨부파일, 알림을 분리한 구조입니다.

채팅 · 메시지 · 파일 ERD

채팅방, 멤버십, 사용자별 목록 메타데이터, 메시지, 첨부파일, 알림을 분리한 구조



설계 포인트

- 채팅방 메타데이터: lastMessage, participantCount 등 목록 표시용 데이터 보관
- 사용자별 상태: chat_list에 lastReadMessageId와 unreadCount 저장
- 참여자 관리: chat_room_member로 role, joined/left, active 관리
- 첨부파일 처리: 선업로드와 orphan cleanup 처리
- 검색: chat_message.message_content Full-Text Index 적용

26. 운영/관리 ERD 및 DDL 검토 메모

관리자 계정, 관리자 로그인 이력, 관리자 액션 로그를 별도 운영 도메인으로 분리했습니다.

admin

- PK id
- UK login_id
- password
- role / status
- last_login_at

action_log

- FK admin_id
- action_type
- target_type/id

login_history

- FK admin_id
- login_at
- success

DDL 검토 결과

치명적인 문법 오류보다는 실제 운영/정합성 측면에서 보완하면 좋은 지점이 보였습니다.
현재 ERD는 업로드한 DDL 기준으로 작성했습니다.

- chat_room ↔ chat_message 순환 FK: 삭제/마이그레이션 시 주의
- created_by / last_sender 삭제 정책: 회원 물리 삭제 정책 명확화 필요
- friend / user_block 자기 자신 방지 CHECK 권장
- login_type.name unique 권장
- admin_login_history는 실패 이력 기록용 attempted_login_id 검토

```
수정 권장 SQL 예시
alter table login_type add constraint uk_login_type_name unique (name);
alter table friend add constraint ck_friend_not_self check (user_id <> friend_user_id);
alter table user_block add constraint ck_user_block_not_self check (blocker_id <> blocked_id);
-- 회원 물리 삭제 허용 시 created_by / last_message_sender_id 삭제 정책 재검토
```